

Proyecto 1 - Búsqueda Adversarial

18 Points Possible

9/21/2026

Intento 1



En progreso

SIGUIENTE: Presentar tarea



Agregar comentario

Unlimited Attempts Allowed

7/29/2026

▼ Detalles

Objetivo

Implementar el juego de Hoppers y un agente inteligente que juegue mediante **Minimax con poda alfa-beta**, **límite de profundidad** y una **función heurística de evaluación**, respetando un tope de **30 segundos por jugada**.

Requisitos

1. Representación e interpretación del juego (funciones estilo Lecture 0)

Deben implementar todas las funciones que definen el problema, análogas a las de los labs anteriores, adaptadas a Hoppers:

- `initial_state()` — el tablero inicial de Hoppers.
- `player(state)` — a qué jugador le toca mover.
- `actions(state)` — el conjunto de todas las jugadas legales desde un estado (incluyendo saltos, y saltos múltiples/encadenados según las reglas de Hoppers).
- `result(state, action)` — el nuevo estado tras aplicar una jugada (sin mutar el estado recibido).
- `terminal(state)` — si la partida terminó.
- `winner(state)` / `utility(state)` — quién ganó y el valor del estado terminal.

Estas funciones deben ser correctas e independientes del agente: cualquier agente debe poder usarlas para jugar.

2. Agente Minimax con poda alfa-beta

- Implementar Minimax con **poda alfa-beta**.
- El agente debe devolver una jugada legal para el jugador en turno en cualquier estado no terminal.

3. Límite de profundidad de búsqueda

- La búsqueda debe cortarse a una **profundidad configurable** (parámetro `depth`).
- Cuando la búsqueda llega al límite de profundidad **sin** alcanzar un estado terminal, debe evaluar la posición con la **función heurística** (de evaluación) en lugar de seguir bajando.

4. Función heurística de evaluación

- Implementar una función que estime el valor de un estado **no terminal** desde la perspectiva del jugador.
- Debe estar documentada: explicar qué mide y por qué.

5. Límite de tiempo: 30 segundos por jugada

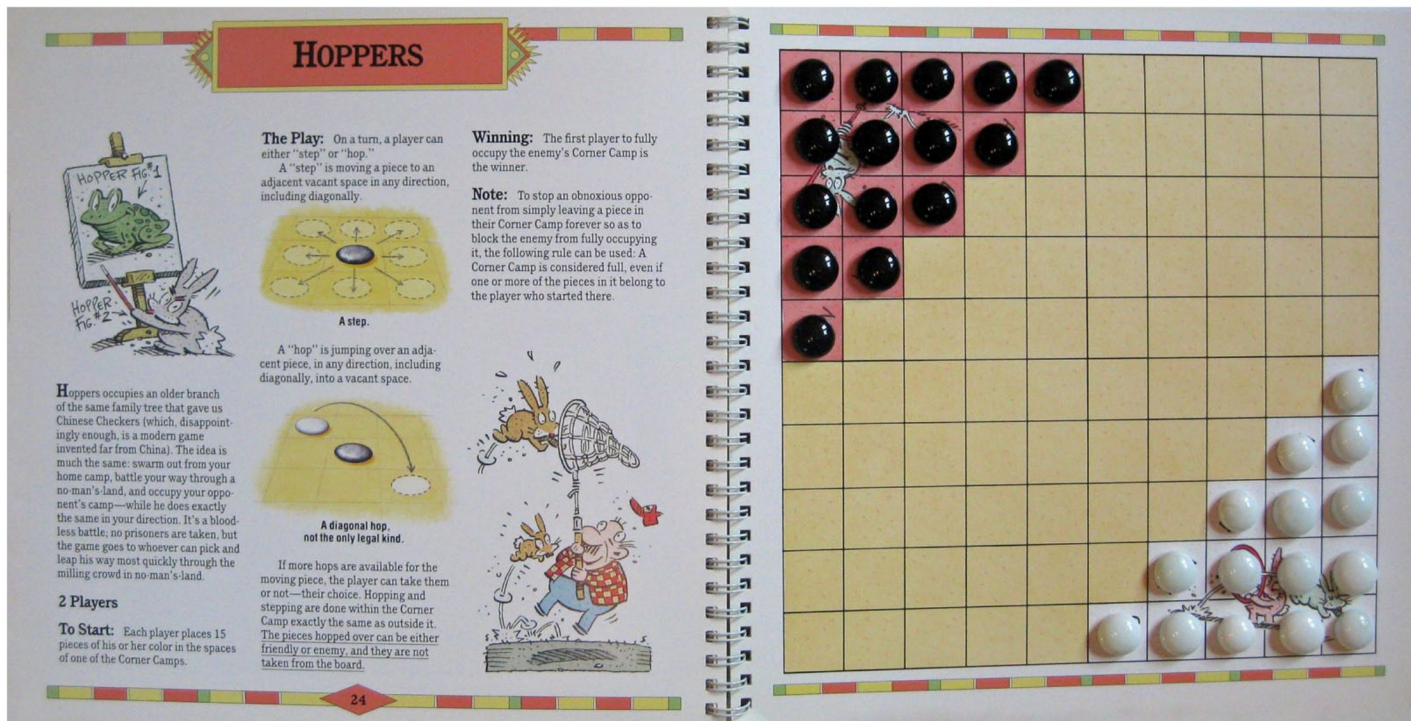
- El agente **no puede tardar más de 30 segundos** en decidir una jugada.
- Se sugiere manejar el tiempo de forma robusta. Cualquier estrategia es válida mientras se respete el tope y siempre se devuelva una jugada legal.

Entregables

- Código con todas las funciones del juego y el agente.
- La función heurística documentada (qué mide y por qué).
- Un modo para **jugar humano vs. agente y agente vs. agente**.
- Un breve informe: descripción de la heurística, cómo se maneja el tiempo y la profundidad, y observaciones.

REPRESENTACIÓN COORDENADAS DEL JUEGO PARA TODOS

	0	1	2	3	4	5	6	7	8	9
0	P1(0 0)	P1(0 1)	P1(0 2)	P1(0 3)	P1(0 4)	0	0	0	0	0
1	P1(1 0)	P1(1 1)	P1(1 2)	P1(1 3)	0	0	0	0	0	0
2	P1(2 0)	P1(2 1)	P1(2 2)	0	0	0	0	0	0	0
3	P1(3 0)	P1(3 1)	0	0	0	0	0	0	0	0
4	P1(4 0)	0	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0	0	P2(5 9)
6	0	0	0	0	0	0	0	0	P2(6 8)	P2(6 9)
7	0	0	0	0	0	0	0	P2(7 7)	P2(7 8)	P2(7 9)
8	0	0	0	0	0	0	P2(8 6)	P2(8 7)	P2(8 8)	P2(8 9)
9	0	0	0	0	0	P2(9 5)	P2(9 6)	P2(9 7)	P2(9 8)	P2(9 9)



Criterios de evaluación

El proyecto es **individual**.

Criterio	Puntos
Funciones del juego correctas (reglas de Hoppers, saltos, estados terminales)	15
Minimax con poda alfa-beta correcto	20
Límite de profundidad funcionando y usando la heurística en el corte	15
Respeto del límite de 30 s por jugada, siempre devolviendo jugada legal	10
El juego termina (cierra) y puede ganar	15
Informe y claridad del código	5
Desempeño del torneo	20
Total	100